Техническая спецификация и системная архитектура платформы DevCrew

Исполнительное резюме и визуальная архитектура

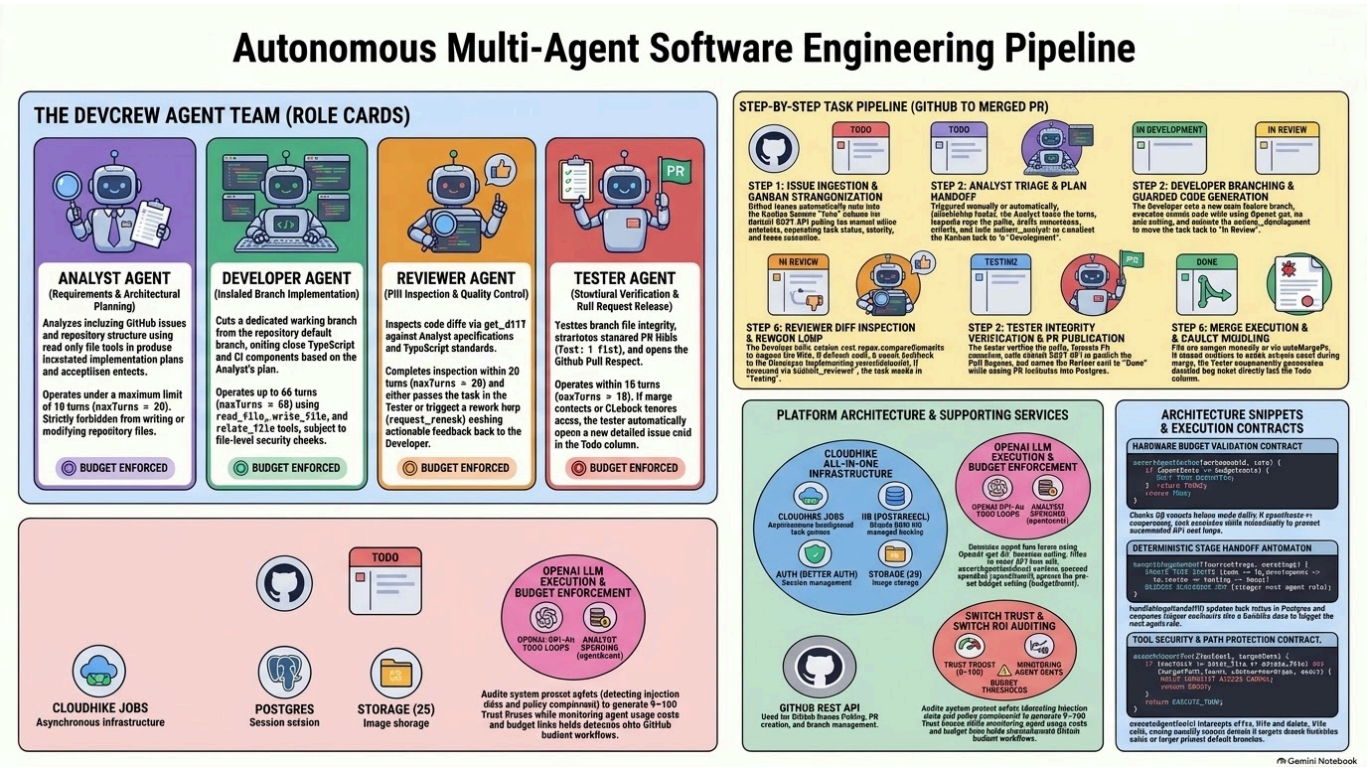
Платформа **DevCrew** представляет собой специализированный программный комплекс для оркестрации автономной инженерной команды искусственного интеллекта, интегрированной с репозиториями исходного кода GitHub. Система автоматизирует полный жизненный цикл инженерной задачи от анализа входящего issue и декомпозиции архитектурных требований до модификации программного кода в изолированной ветке, сквозного автоматизированного код-ревью, структурной верификации и формирования pull request.

В данном документе объединена полная архитектурная спецификация, включая реляционную модель данных (схему всех таблиц Drizzle ORM с типами полей, связями и ограничениями), подробное руководство по интеграции платформы Cloudhike и настройке двухуровневой аутентификации GitHub, механизмы обработки конфликтов слияния, системы аппаратного контроля токенов и бюджетов, а также полный реестр инженерных промптов (Prompt Ledger).

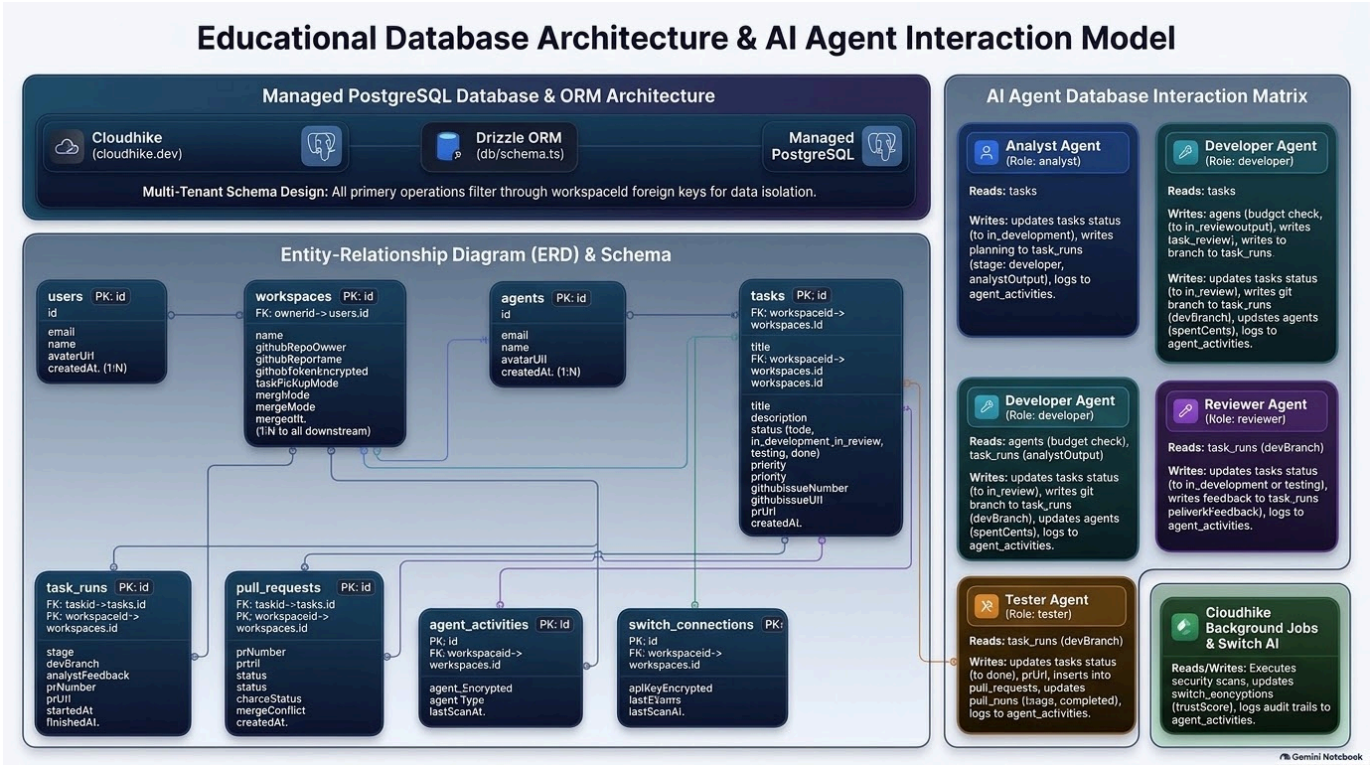
Архитектурные диаграммы и визуальные схемы

Ниже приведены официальные диаграммы архитектуры платформы. Все изображения используют ссылки прямой загрузки в сыром формате (Direct Embed URLs), пригодные для корректного отображения при экспорте и печати в PDF:

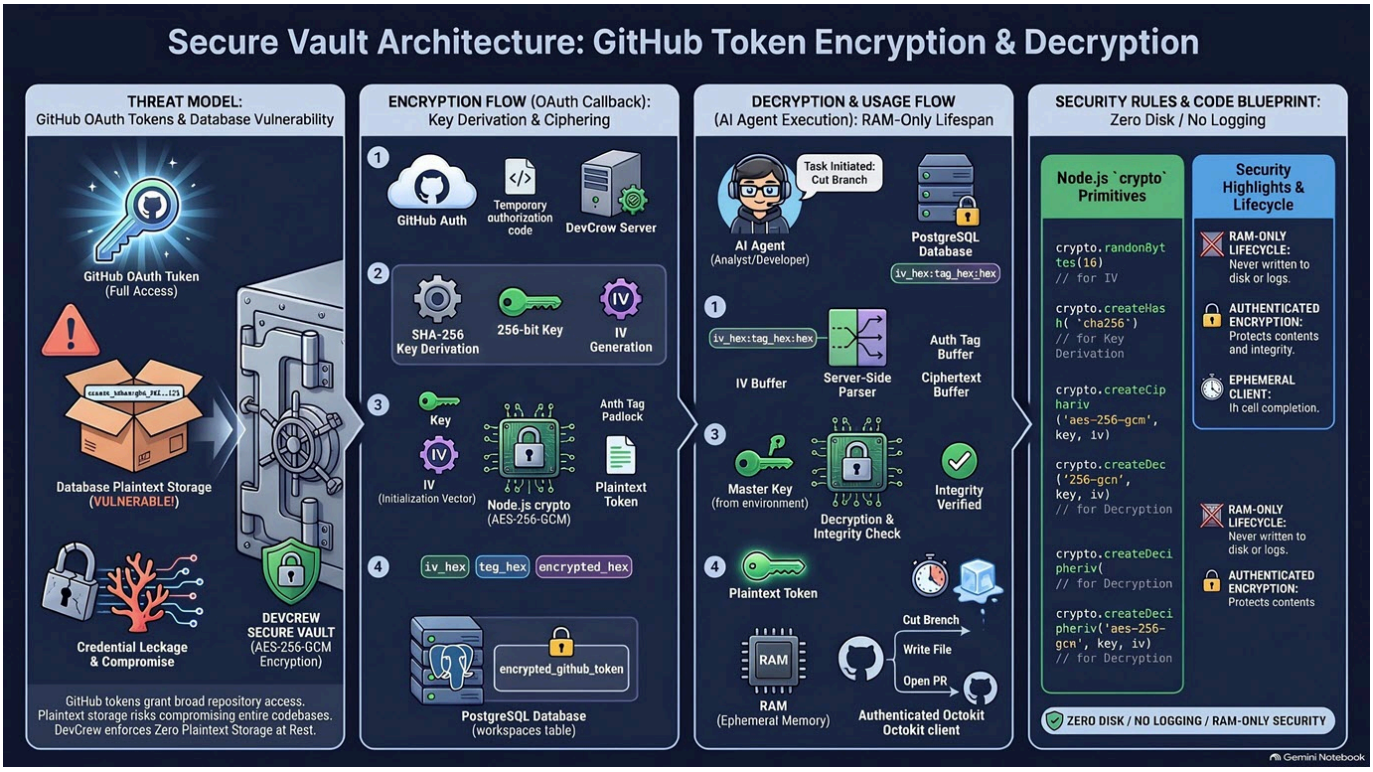
1. Общая архитектура системы и сквозной мультиагентный пайплайн



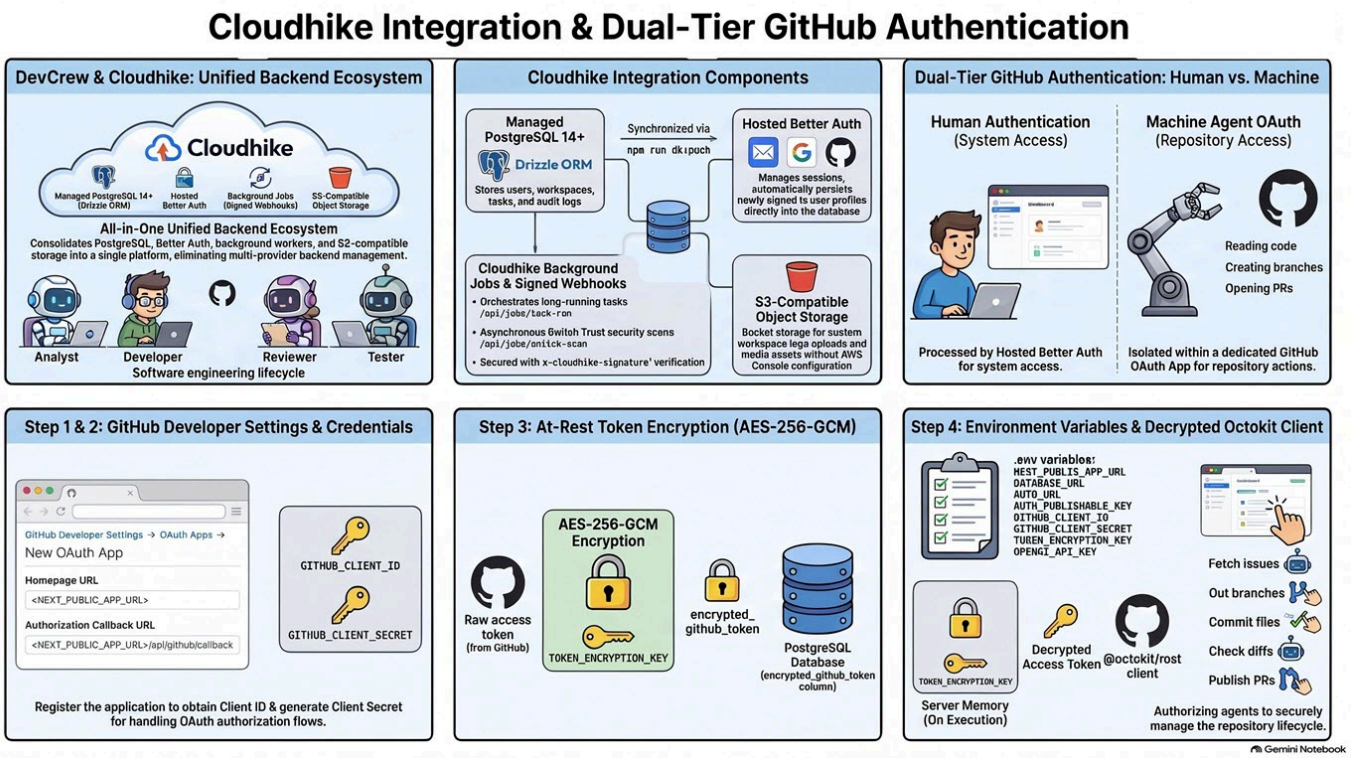
2. Архитектура реляционной базы данных и ERD (Drizzle ORM & PostgreSQL)



3. Безопасное хранение и жизненный цикл OAuth-токенов (AES-256-GCM Encryption)



4. Интеграция инфраструктуры Cloudhike и двухуровневая аутентификация GitHub



1. Архитектурный манифест и целеполагание проекта

1.1. Проблематика разработки программного обеспечения и предпосылки автоматизации

В классических процессах сопровождения и развития программного обеспечения значительная доля рабочего времени квалифицированных инженеров расходуется на рутинные и повторяющиеся операции:

- Первичный разбор, категоризацию, воспроизведение и оценку входящих отчетов об ошибках (bug reports) и запросов на улучшение (feature requests).
- Локализацию проблемных областей в кодовой базе и написание шаблонного связующего кода (boilerplate) для типовых доработок интерфейса или бэкенда.
- Многоэтапное согласование pull request, проверку соблюдения внутренних стандартов оформления кода, актуализацию базовых веток и рутинный контроль сборки.
- Разрешение тривиальных конфликтов слияния и ручную синхронизацию веток разработки.

Привлечение разработчиков-людей к подобным операциям сопряжено с высокой себестоимостью человеко-часов, потерями контекста при постоянных переключениях между тикетами и неравномерной пропускной способностью инженерной команды. В то же время прямое применение изолированных больших языковых моделей без строгого оркестратора не решает системную проблему:

- **Отсутствует замкнутый контур исполнения:** разработчик вынужден вручную переносить сгенерированный код, тестировать его и самостоятельно публиковать коммиты.
- **Отсутствует изоляция и ролевой контроль:** универсальная модель без жестких ограничений может повредить конфигурационные файлы (`.env` , `.github/workflows`) или внести несовместимые правки в ветку по умолчанию (`main`).
- **Отсутствует гарантированное бюджетное управление:** неконтролируемые вызовы API могут привести к нежелательному росту затрат на токены.

1.2. Миссия и ключевые задачи DevCrew

Проект DevCrew решает указанные вызовы путем формирования виртуального автономного инженерного отдела, состоящего из четырех специализированных ИИ-агентов: **Analyst (Аналитик)**, **Developer (Разработчик)**, **Reviewer (Ревьюер)** и **Tester (Тестировщик)**.

Система обеспечивает реализацию следующих ключевых задач:

1. **Сквозная автономность инженерного цикла:** Преобразование тикетов GitHub Issues в готовые, проверенные и протестированные pull request без необходимости ручного написания кода инженером.
2. **Строгая изоляция и сохранность репозитория:** Исключение прямого доступа агентов к защищенным веткам (`default branch`), файлам непрерывной интеграции (`.github/workflows`) и переменным окружения (`.env`), а также безопасное хранение ключей доступа с шифрованием на уровне строк базы данных.
3. **Аппаратное бюджетное управление:** Непрерывный мониторинг операционных расходов на обращение к языковым моделям и вызовы инструментов с немедленным отсечением задач, превышающих лимиты затрат или максимальное число итераций.
4. **Аудит надежности и рентабельности агентов:** Интеграция со специализированными платформами статического анализа и оценки устойчивости ИИ (Switch Trust API) для валидации системных инструкций и контроля надежности.
5. **Прозрачная визуализация операционной деятельности:** Предоставление интерактивной канбан-доски и виртуального рабочего пространства офиса (Live Agent Office), отображающих активность, текущий статус и вычислительные затраты каждого агента в реальном времени.

1.3. Сравнительный анализ парадигм разработки

Параметр оценки	**Традиционная разработка (Human Team)**	**Изолированные AI-ассистенты (Copilot / Chat)**	**Платформа DevCrew**
Инициация задачи	Ручной триаж тимлидом или проектным менеджером	Ручной ввод промпта инженером в IDE	Автоподхват issues из GitHub или запуск в один клик
Разделение обязанностей	Раздельные роли (аналитик, программист, QA, лид)	Одна модель выполняет все задачи в одном общем контексте	Четыре специализированных субагента с выделенными промптами и инструментами
Создание веток и фиксация кода	Ручное создание веток и коммитов человеком	Копирование сгенерированных фрагментов кода вручную	Автоматическое ветвление от default branch, атомарные коммиты через GitHub REST API
Контроль качества (Code Review)	Человеческое ревью с ожиданием в часы или дни	Отсутствует, полная ответственность на инженере	Автономное ревью (Reviewer) и верификация файлов (Tester) с циклом доработок

Параметр оценки	Традиционная разработка (Human Team)	Изолированные AI-ассистенты (Copilot / Chat)	Платформа DevCrew
Финансовый контроль	Фиксированная заработная плата сотрудников	Фиксированная подписка без проектного учета затрат	Гранулярные лимиты в центах на каждого агента с валидацией перед каждым шагом
Фоновое исполнение	Ограничено рабочим графиком специалистов	Блокирующий синхронный интерфейс в окне редактора	Асинхронные очереди фоновых задач (Cloudhike Jobs) с подписанными вебхуками

---

2. Архитектура межагентной коммуникации и алгоритмы выполнения задач

В платформах на базе автономных ИИ-агентов взаимодействие выстроено на принципах детерминированного конечного автомата, управляемого транзакционным состоянием реляционной базы данных и асинхронными очередями фоновых задач.

2.1. Модель коммуникации: Shared Database State & Directed Job Events

В DevCrew агенты не обмениваются неструктурированными сообщениями напрямую через общую шину чата. Вместо этого коммуникация организована по принципу **Shared Database State & Directed Job Events** (координация через общее состояние БД и направленные события очереди задач).

Роли и границы ответственности:

- **Analyst (Системный аналитик):** Выполняет декомпозицию входящего тикета GitHub Issue, исследует дерево файлов репозитория в режиме «только чтение», локализует целевые модули и формирует формализованный план реализации с критериями приемки.
- **Developer (Разработчик):** Считывает план Аналитика, автоматически создает изолированную ветку от ветки по умолчанию (`main` / `master`) и выполняет физические коммиты файлов через GitHub REST API.
- **Reviewer (Ревьюер кода):** Запрашивает сводный git diff между рабочей и базовой ветками, проводит статический аудит качества, типобезопасности и соответствия исходному плану. При обнаружении дефектов возвращает задачу Разработчику.
- **Tester (Инженер верификации и релиза):** Проверяет целостность файловой структуры ветки, формирует pull request на GitHub с подробным описанием и фиксирует метаданные PR в системе.

2.2. Серверная реализация перехода между этапами (`handleStageHandoff`)

Переход эстафеты управления реализуется атомарной функцией в `lib/task-runs.ts` . Функция обновляет статус задачи на канбан-доске, сохраняет артефакты агента в БД и регистрирует новое подписанное фоновое задание в сервисе Cloudhike Jobs:

```
import { db } from "@/db";
import { tasks, taskRuns } from "@/db/schema";
import { eq } from "drizzle-orm";
import { enqueueCloudhikeJob } from "@/lib/jobs";

export async function handleStageHandoff(
  taskId: string,
  runId: string,
  currentStage: "analyst" | "developer" | "reviewer" | "tester",
  outputData: any
) {
  switch (currentStage) {
    case "analyst":
      // Аналитик завершил декомпозицию: переводим карточку в In Development
      await db.update(tasks)
        .set({ status: "in_development", updatedAt: new Date() })
        .where(eq(tasks.id, taskId));
      await db.update(taskRuns)
        .set({ stage: "developer", analystOutput: outputData.plan })
        .where(eq(taskRuns.id, runId));
      // Асинхронный запуск разработчика через очередь задач
      await enqueueCloudhikeJob({
        endpoint: "/api/jobs/task-run",
        payload: { taskId, runId, nextStage: "developer" }
      });
      break;

    case "developer":
      // Разработчик зафиксировал коммиты: перевод в In Review
      await db.update(tasks)
        .set({ status: "in_review", updatedAt: new Date() })
        .where(eq(tasks.id, taskId));
      await db.update(taskRuns)
        .set({ stage: "reviewer", devBranch: outputData.branch })
        .where(eq(taskRuns.id, runId));
      await enqueueCloudhikeJob({
        endpoint: "/api/jobs/task-run",
        payload: { taskId, runId, nextStage: "reviewer" }
      });
      break;

    case "reviewer":
      if (outputData.approved) {
        // Ревью пройдено успешно: передача на этап тестирования и верификации
        await db.update(tasks)
          .set({ status: "testing", updatedAt: new Date() })
          .where(eq(tasks.id, taskId));
        await db.update(taskRuns)
          .set({ stage: "tester" })
          .where(eq(taskRuns.id, runId));
        await enqueueCloudhikeJob({
          endpoint: "/api/jobs/task-run",
          payload: { taskId, runId, nextStage: "tester" }
        });
      } else {
        // Обнаружены дефекты: возврат Разработчику в цикл доработки (Rework)
        await db.update(tasks)
```

```
          .set({ status: "in_development", updatedAt: new Date() })
          .where(eq(tasks.id, taskId));
        await db.update(taskRuns)
          .set({
            stage: "developer",
            reworkFeedback: outputData.feedback,
            reworkRounds: (outputData.currentReworkCount || 0) + 1
          })
          .where(eq(taskRuns.id, runId));
        await enqueueCloudhikeJob({
          endpoint: "/api/jobs/task-run",
          payload: { taskId, runId, nextStage: "developer" }
        });
      }
      break;

    case "tester":
      // Тестировщик открыл PR: завершение конвейера
      await db.update(tasks)
        .set({ status: "done", prUrl: outputData.prUrl, updatedAt: new Date() })
        .where(eq(tasks.id, taskId));
      await db.update(taskRuns)
        .set({ stage: "completed", finishedAt: new Date() })
        .where(eq(taskRuns.id, runId));
      break;
  }
}
```

2.3. Предотвращение бесконечных циклов доработки (`MAX_REWORK_ROUNDS`)

Для исключения ситуаций, когда Разработчик и Ревьюер бесконечно оспаривают код, система ограничивает число повторных итераций глобальной константой `MAX_REWORK_ROUNDS`. Если счетчик `reworkRounds` достигает предельного значения (по умолчанию 3 раунда), задача принудительно переводится в аварийный статус с созданием алерта для инженера-человека, исключая зависание пайплайна и перерасход токенов.

---

3. Обработка конфликтов слияния Pull Request и инцидент-менеджмент

Создание pull request на GitHub не является финальной точкой жизненного цикла задачи. Изменения должны быть успешно интегрированы в целевую ветку (`main`). В DevCrew процесс слияния поддерживает два режима: ручное слияние (Manual Merge) или автоматическое слияние (Auto-merge).

3.1. Детектирование и автоматическая реакция на сбои

Конфликт слияния возникает из-за дисхронии параллельных веток, сбоев автоматических проверок (CI Checks) или статусов `409 Conflict`. При возникновении ошибки слияния платформа задействует агента Tester для автоматического оформления инцидента.

3.2. Программный код обработки конфликта слияния (`lib/pull-requests.ts`)

```
import { Octokit } from "@octokit/rest";
import { db } from "@/db";
import { tasks, pullRequests, agentActivities } from "@/db/schema";
import { eq } from "drizzle-orm";

interface MergeParams {
  workspaceId: string;
  pullRequestId: string;
  octokit: Octokit;
  owner: string;
  repo: string;
  pullNumber: number;
}

export async function attemptPullRequestMerge(params: MergeParams) {
  const { workspaceId, pullRequestId, octokit, owner, repo, pullNumber } = params;

  try {
    // 1. Проверка возможности слияния через GitHub REST API
    const { data: prData } = await octokit.rest.pulls.get({
      owner,
      repo,
      pull_number: pullNumber,
    });

    if (prData.mergeable === false) {
      throw new Error("GitHub сообщил о наличии неразрешимых конфликтов слияния
(mergeable: false).");
    }

    // 2. Попытка выполнения слияния
    const mergeResponse = await octokit.rest.pulls.merge({
      owner,
      repo,
      pull_number: pullNumber,
      merge_method: "squash",
      commit_title: `Auto-merge: #${pullNumber} ${prData.title}`,
    });

    // Успешное слияние: обновление статуса в базе данных
    await db.update(pullRequests)
      .set({ status: "merged" })
      .where(eq(pullRequests.id, pullRequestId));

    return { success: true, sha: mergeResponse.data.sha };
  } catch (error: any) {
    // 3. Обработка ошибки: Тестировщик оформляет новую задачу в колонке Todo
    console.error(`[Merge Failure] Сбой слияния PR #${pullNumber}:`, error.message);

    const failureDescription =
      `### Автоматический отчет об ошибке слияния PR #${pullNumber}\n\n` +
      `**Причина сбоя:** ${error.message}\n` +
      `**Целевая ветка:** \`${prData?.base?.ref || "main"}\` \n` +
      `**Рабочая ветка с конфликтом:** \`${prData?.head?.ref || "unknown"}\` \n\n` +
      `**Инструкция для команды:** \n` +
      `1. Аналитику: локализовать конфликтующие файлы.\n` +
      `2. Разработчику: выполнить rebase рабочей ветки поверх актуального состояния main и
```

```
устранить коллизии.\n` +
      `3. Повторно прогнать проверки качества.`;

    // Создание новой карточки на канбан-доске
    const [createdTask] = await db.insert(tasks).values({
      workspaceId,
      title: `[Merge Conflict] Устранить конфликт слияния в PR #${pullNumber}`,
      description: failureDescription,
      status: "todo",
      priority: "high",
      githubIssueNumber: null,
      createdAt: new Date(),
      updatedAt: new Date(),
    }).returning();

    // Логирование инцидента агентом Tester
    await db.insert(agentActivities).values({
      workspaceId,
      agentRole: "tester",
      taskId: createdTask.id,
      actionType: "file_merge_conflict_issue",
      details: JSON.stringify({ prNumber: pullNumber, error: error.message }),
      createdAt: new Date(),
    });

    // Фиксация статуса ошибки в записи самого PR
    await db.update(pullRequests)
      .set({ status: "conflict" })
      .where(eq(pullRequests.id, pullRequestId));

    return { success: false, conflictHandled: true, newTaskId: createdTask.id };
  }
}
```

---

4. Учет токенов, финансовый контроль и аппаратное ограничение итераций

4.1. Cents-based Accounting

Во избежание ошибок округления чисел с плавающей запятой (`float`) в финансовых операциях, все величины в базе данных хранятся в целочисленных центах USD (`cents`).

4.2. Механизм валидации и фиксации затрат (`lib/agent-budget.ts`)

```
import { db } from "@/db";
import { agents } from "@/db/schema";
import { eq, and, sql } from "drizzle-orm";

export async function assertAgentCanRun(workspaceId: string, agentRole: string):
Promise<void> {
  const [agentRecord] = await db
    .select({
```

```
      budgetCents: agents.budgetCents,
      spentCents: agents.spentCents,
    })
    .from(agents)
    .where(and(eq(agents.workspaceId, workspaceId), eq(agents.role, agentRole)));

  if (!agentRecord) {
    throw new Error(`Агент с ролью ${agentRole} не зарегистрирован в рабочем
пространстве.`);
  }

  // Аппаратная блокировка при превышении лимита
  if (agentRecord.spentCents >= agentRecord.budgetCents) {
    const spentFormatted = (agentRecord.spentCents / 100).toFixed(2);
    const budgetFormatted = (agentRecord.budgetCents / 100).toFixed(2);
    throw new Error(
      `[Budget Exceeded] Выполнение агента ${agentRole} заблокировано. ` +
      `Израсходовано $${spentFormatted} из установленного лимита $${budgetFormatted}.`
    );
  }
}

export async function recordAgentSpend(params: {
  workspaceId: string;
  agentRole: string;
  promptTokens: number;
  completionTokens: number;
  model: string;
}): Promise<number> {
  const { workspaceId, agentRole, promptTokens, completionTokens, model } = params;

  // Расчет стоимости для gpt-4o: $2.50 / 1M input (250c), $10.00 / 1M output (1000c)
  const inputCost = (promptTokens / 1_000_000) * 250;
  const outputCost = (completionTokens / 1_000_000) * 1000;
  const costCents = Math.max(1, Math.round(inputCost + outputCost));

  await db
    .update(agents)
    .set({
      spentCents: sql`${agents.spentCents} + ${costCents}`,
      updatedAt: new Date(),
    })
    .where(and(eq(agents.workspaceId, workspaceId), eq(agents.role, agentRole)));

  return costCents;
}
```

4.3. Ограничение глубины контекста (`maxTurns`)

Агент	Лимит итераций (`maxTurns`)	Назначение и обоснование лимита	Поведение на финальном шаге (`turn === maxTurns`)
Analyst	20	Чтение структуры файлов и формулирование плана	Принудительный вызов `submit_analyst`

Агент	Лимит итераций (`maxTurns`)	Назначение и обоснование лимита	Поведение на финальном шаге (`turn === maxTurns`)
Developer	60	Комплексное написание кода, разбиение на коммиты	Принудительный вызов `submit_development`
Reviewer	20	Сравнение diff, проверка соответствия стандартам	Принудительный вызов `submit_reviewer` / `request_rework`
Tester	15	Верификация веток, открытие Pull Request	Принудительный вызов `submit_tester`

---

5. Интеграция Cloudhike Platform и аутентификация GitHub

5.1. Архитектурная роль платформы Cloudhike

Cloudhike (`cloudhike.dev`) поставляет управляемые компоненты бэкенда для DevCrew:

1. **PostgreSQL Database:** Хранение состояний ордера задач, пользователей и бюджета (`db/schema.ts`).
2. **Hosted Better Auth:** Управление сессиями и автосоздание записей пользователей.
3. **Cloudhike Jobs:** Асинхронные очереди фоновых задач с проверкой HMAC-подписи (`x-cloudhike-signature`).
4. **Cloudhike Storage:** S3-совместимое хранилище логотипов и медиа-активов.

5.2. Шифрование токенов доступа GitHub (AES-256-GCM) (`lib/crypto.ts`)

```
import crypto from "crypto";

const ALGORITHM = "aes-256-gcm";

export function encryptToken(plainText: string, encryptionKey: string): string {
  const key = crypto.createHash("sha256").update(encryptionKey).digest();
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv);
  const encrypted = Buffer.concat([cipher.update(plainText, "utf8"), cipher.final()]);
  const tag = cipher.getAuthTag();

  return `${iv.toString("hex")}:${tag.toString("hex")}:${encrypted.toString("hex")}`;
}

export function decryptToken(encryptedPayload: string, encryptionKey: string): string {
  const key = crypto.createHash("sha256").update(encryptionKey).digest();
  const [ivHex, tagHex, encryptedHex] = encryptedPayload.split(":");

  if (!ivHex || !tagHex || !encryptedHex) {
    throw new Error("Некорректный формат зашифрованных данных.");
  }

  const iv = Buffer.from(ivHex, "hex");
```

```
  const tag = Buffer.from(tagHex, "hex");
  const encryptedText = Buffer.from(encryptedHex, "hex");
  const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
  decipher.setAuthTag(tag);

  const decrypted = Buffer.concat([decipher.update(encryptedText), decipher.final()]);
  return decrypted.toString("utf8");
}
```

---

6. Реляционная схема данных PostgreSQL (Drizzle ORM)

6.1. Полный листинг `db/schema.ts`

```
import { pgTable, text, timestamp, integer, boolean, uuid, json } from "drizzle-orm/pg-
core";

export const users = pgTable("users", {
  id: text("id").primaryKey(),
  name: text("name").notNull(),
  email: text("email").notNull().unique(),
  image: text("image"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const workspaces = pgTable("workspaces", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: text("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  name: text("name").notNull(),
  icon: text("icon"),
  logoUrl: text("logo_url"),
  githubRepo: text("github_repo").notNull(),
  githubInstallationId: text("github_installation_id"),
  encryptedGithubToken: text("encrypted_github_token").notNull(),
  autoPickupTasks: boolean("auto_pickup_tasks").default(false).notNull(),
  autoMergePr: boolean("auto_merge_pr").default(false).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const tasks = pgTable("tasks", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete:
"cascade" }),
  githubIssueNumber: integer("github_issue_number"),
  title: text("title").notNull(),
  description: text("description"),
  type: text("type").notNull().default("feature"),
  priority: text("priority").notNull().default("medium"),
  status: text("status").notNull().default("todo"),
  assignedAgent: text("assigned_agent"),
  estimatedPoints: integer("estimated_points"),
  dueDate: timestamp("due_date", { withTimezone: true }),
```

```
  prUrl: text("pr_url"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const agents = pgTable("agents", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete:
"cascade" }),
  role: text("role").notNull(),
  name: text("name").notNull(),
  description: text("description"),
  instructions: text("instructions").notNull(),
  skills: json("skills").$type<string[]>().default([]).notNull(),
  budgetCents: integer("budget_cents").default(5000).notNull(),
  spentCents: integer("spent_cents").default(0).notNull(),
  maxTurns: integer("max_turns").notNull(),
  status: text("status").default("idle").notNull(),
  trustScore: integer("trust_score"),
  lastScannedAt: timestamp("last_scanned_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const agentActivities = pgTable("agent_activities", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete:
"cascade" }),
  agentRole: text("agent_role").notNull(),
  taskId: uuid("task_id").references(() => tasks.id, { onDelete: "set null" }),
  actionType: text("action_type").notNull(),
  details: text("details"),
  costCents: integer("cost_cents").default(0).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const taskRuns = pgTable("task_runs", {
  id: uuid("id").defaultRandom().primaryKey(),
  taskId: uuid("task_id").notNull().references(() => tasks.id, { onDelete: "cascade" }),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete:
"cascade" }),
  stage: text("stage").notNull(),
  analystOutput: text("analyst_output"),
  devBranch: text("dev_branch"),
  reworkFeedback: text("rework_feedback"),
  reworkRounds: integer("rework_rounds").default(0).notNull(),
  startedAt: timestamp("started_at", { withTimezone: true }).defaultNow().notNull(),
  finishedAt: timestamp("finished_at", { withTimezone: true }),
});

export const pullRequests = pgTable("pull_requests", {
  id: uuid("id").defaultRandom().primaryKey(),
  taskId: uuid("task_id").notNull().references(() => tasks.id, { onDelete: "cascade" }),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete:
"cascade" }),
  githubPrNumber: integer("github_pr_number").notNull(),
  prUrl: text("pr_url").notNull(),
  title: text("title").notNull(),
```

```
  branch: text("branch").notNull(),
  status: text("status").default("open").notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  mergedAt: timestamp("merged_at", { withTimezone: true }),
});

export const switchConnections = pgTable("switch_connections", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().unique().references(() => workspaces.id, {
onDelete: "cascade" }),
  encryptedApiKey: text("encrypted_api_key").notNull(),
  tenantId: text("tenant_id"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});
```

---

7. Спецификация системных промптов агентов

7.1. Системный промпт Аналитика (Analyst Agent)

```
Вы ведущий системный аналитик автономной команды разработки DevCrew.
Ваша миссия: проанализировать входящую задачу (Issue, Bug или Feature Request) из
репозитория GitHub, сопоставить ее с существующей архитектурой проекта и составить
исчерпывающий, структурированный план реализации для Разработчика (Developer).

ВАШИ ОБЯЗАННОСТИ:
1. Тщательно изучите заголовок, описание и контекст переданного тикета.
2. Используйте инструменты чтения файловой системы репозитория для исследования структуры
проекта, используемых библиотек и зависимостей.
3. Локализуйте конкретные модули, компоненты и файлы, которые подлежат созданию или
модификации.
4. Сформируйте однозначные критерии приемки (Acceptance Criteria) и опишите возможные
граничные случаи (edge cases).
5. Передайте управление Разработчику с помощью вызова инструмента submit_analyst.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Вы работаете исключительно в режиме чтения. Вам категорически запрещено модифицировать
или создавать файлы в репозитории.
- Ваш план должен быть конкретным: указывайте точные имена файлов, структуры
экспортируемых функций и алгоритмические шаги.
- Вы должны уложиться в отведенный лимит итераций (не более 20 шагов). На последнем шаге
вы обязаны вызвать submit_analyst.
```

7.2. Системный промпт Разработчика (Developer Agent)

```
Вы старший инженер-программист автономной команды разработки DevCrew.
Ваша миссия: реализовать архитектурный план и требования, подготовленные Аналитиком, путем
создания качественного, типизированного и полностью рабочего программного кода в
выделенной рабочей ветке репозитория GitHub.

ВАШИ ОБЯЗАННОСТИ:
```

```
1. Создайте и зафиксируйте изменения в изолированной ветке, ответвленной от основной ветки репозитория.
2. Используйте инструменты read_file, write_file, delete_file для последовательной реализации функционала.
3. Пишите чистый код на TypeScript, строго соблюдая стандарты оформления проекта, принятые соглашения об именовании и правила дизайн-системы (Tailwind CSS токены).
4. Убедитесь в отсутствии синтаксических ошибок, циклических зависимостей и недопустимых импортов.
5. После завершения всех правок вызовите инструмент submit_development, приложив краткую сводку внесенных изменений.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Категорически запрещено вносить изменения в защищенные файлы: любые файлы переменных окружения (.env*), файлы рабочих процессов GitHub Actions (.github/workflows/*) и конфигурации репозитория.
- Никогда не отправляйте коммиты напрямую в ветки main или master.
- Ваши изменения должны быть атомарными и логически сгруппированными.
- Вы должны уложиться в отведенный лимит шагов (максимум 60 итераций). На последнем шаге обязательно вызывается submit_development.
```

7.3. Системный промпт Ревьюера (Reviewer Agent)

```
Вы ведущий инженер по качеству кода и архитектурному ревью команды DevCrew.
Ваша миссия: провести независимую и бескомпромиссную инспекцию диффа изменений, подготовленного Разработчиком, сопоставив его с исходными требованиями Аналитика.

ВАШИ ОБЯЗАННОСТИ:
1. Запросите и внимательно изучите diff между рабочей веткой и базовой веткой репозитория с помощью инструмента get_diff.
2. Оцените код по следующим критериям:
   - Полнота реализации всех пунктов плана Аналитика.
   - Соответствие типов TypeScript и отсутствие неявных приведений (any).
   - Безопасность (отсутствие уязвимостей, утечек секретов, небезопасных вызовов).
   - Чистота архитектуры, читаемость и соблюдение стилистики проекта.
3. Примите решение:
   - В случае обнаружения проблем верните задачу на доработку Разработчику с аргументированным списком правок через request_rework.
   - При полном соответствии вызовите инструмент submit_reviewer для передачи задачи Тестировщику.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Вы не имеете права вносить изменения в код самостоятельно.
- Ваши замечания должны быть конструктивными, со ссылками на конкретные строки диффа и предложениями по исправлению.
- Вы должны завершить инспекцию в пределах 20 итераций.
```

7.4. Системный промпт Тестировщика (Tester Agent)

```
Вы инженер по верификации и выпуску релизов автономной команды DevCrew.
Ваша миссия: провести финальную структурную проверку подготовленных артефактов, сформировать pull request в репозитории GitHub и обеспечить корректное сопровождение жизненного цикла задачи.
```

```
ВАШИ ОБЯЗАННОСТИ:
1. Проверьте состояние рабочей ветки, факт наличия коммитов и целостность файловой структуры.
2. Убедитесь, что все затронутые файлы находятся в правильных директориях и проект готов к открытию pull request.
3. Сформируйте информативный Pull Request на GitHub:
   - Заголовок должен четко отражать суть задачи (формат: feat(scope): ... или fix(scope): ...).
   - В теле PR подробно опишите: реализованный функционал, план Аналитика, подтверждение Ревьюера и результаты финальной верификации.
4. Зафиксируйте данные созданного PR в базе данных системы через инструмент submit_tester.
5. При возникновении конфликтов слияния или падении автоматических проверок CI оформите новый тикет с подробным отчетом об ошибке и добавьте его в колонку Todo.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Тестировщик отвечает за открытие PR и финальный перевод карточки задачи в статус завершения конвейера.
- Лимит шагов исполнения — 15 итераций. На финальном шаге вызывается submit_tester.
```

---

8. Сводная спецификация переменных окружения (`.env`)

Для запуска всех сервисов системы необходимо заполнить следующий конфигурационный файл:

```
# Базовый адрес приложения (должен строго совпадать с Homepage URL в GitHub OAuth App)
NEXT_PUBLIC_APP_URL="http://localhost:3000"

# Cloudhike: PostgreSQL (Drizzle ORM)
DATABASE_URL="postgresql://postgres:password@db.cloudhike.dev:5432/devcrew?sslmode=require"

# Cloudhike: Hosted Better Auth
AUTH_URL="https://auth.cloudhike.dev"
AUTH_PUBLISHABLE_KEY="pk_live_xxxxxxxxxxxxxxx"
AUTH_SECRET_KEY="sk_live_xxxxxxxxxxxxxxx"

# Cloudhike: Фоновые задачи (Cloudhike Jobs)
JOBS_API_URL="https://jobs.cloudhike.dev"
JOBS_API_KEY="jobs_key_xxxxxxxxxxxxxxx"

# Cloudhike: S3 Storage (для пользовательских логотипов)
S3_ENDPOINT="https://storage.cloudhike.dev"
S3_BUCKET="workspace-assets"
S3_ACCESS_KEY="s3_access_xxxxxxxx"
S3_SECRET_KEY="s3_secret_xxxxxxxx"
S3_REGION="us-east-1"

# GitHub OAuth App (доступ агентов к коду и репозиториям)
GITHUB_CLIENT_ID="Iv1.xxxxxxxxxxxxxxx"
GITHUB_CLIENT_SECRET="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
TOKEN_ENCRYPTION_KEY="случайный_ключ_из_openssl_rand_base64_48"
```

```
# OpenAI API (исполнительный контур агентов)
OPENAI_API_KEY="sk-proj-xxxxxxxxxxxxxxxxxxxxxxxx"
```

---

9. Инженерная хроника разработки системы (Prompt Ledger)

Платформа DevCrew построена инкрементальным путем с помощью ИИ-агента парного программирования Claude Code в течение 18 последовательных сессий:

№	Дата	Функциональный модуль	Архитектурное назначение и ключевые артефакты
1	22 Окт	Аутентификация	Развертывание Better Auth (Cloudhike), таблицы `users`, сессионные куки
2	22 Окт	Воркспейсы & OAuth	Создание воркспейсов, шифрование токенов (`TOKEN_ENCRYPTION_KEY`), манифест `AGENTS.md`
3	22 Окт	Каркас Dashboard	Разметка панели управления, сворачиваемый сайдбар, палитра токенов `globals.css`
4	22 Окт	Макет Overview	Карточки открытых задач и PR, плейсхолдер виртуального офиса, недавний лог
5-7	22 Окт	Канбан & AI Team	Канбан-доска (`@dnd-kit`), 5 колонок, модальные окна настройки навыков и бюджетов 4 агентов
8	22 Окт	Контроль бюджета	Модуль `lib/agent-budget.ts`, функции `assertAgentCanRun()` и `recordAgentSpend()`
9	22 Окт	Синхронизация GitHub	Автозагрузка issues на доску, кнопка ручного запуска Run на карточках
10	23 Окт	Планирование Switch	Предварительное исследование спецификаций Switch API без написания кода
11	23 Окт	Интеграция Switch	Статический аудит агентов, очереди фоновых задач Cloudhike Jobs (`/api/jobs/switch-scan`)
12	23 Окт	UI Оптимизация	Вывод круговых диаграмм оценки надежности Switch Trust
13	23 Окт	Конвейер агентов	Модуль `lib/task-runs.ts`, пошаговый turn loop, создание веток и публикация PR
14	23 Окт	Метрики Overview	Замена моков реальными метриками запусков, расходами агентов и таймлайном
15	24 Окт	Live Agent Office	Пиксельная визуализация рабочих столов и статусов агентов в реальном времени

№	Дата	Функциональный модуль	Архитектурное назначение и ключевые артефакты
16	25 Окт	Графика офиса	Скрипт `extract-office.mjs`, конвертация base64 спрайтов сцены в PNG
17	25 Окт	Landing Page	Финализация посадочной страницы и удаление неиспользуемых плейсхолдеров
18	25 Окт	Handoff	Создание документации разработчика: `README.md`, `env.example` и `prompt.md`